

مستند نگاشت و انتقال داده‌ها (Olist ETL Mapping)

پروژه: Olist E-Commerce Data Warehouse

پلتفرم: Microsoft SQL Server / T-SQL

دامنه‌ی داده: سفارش‌های ثبت‌شده در بازه‌ی ۲۰۱۶ تا ۲۰۱۸ (دیتاست Kaggle)

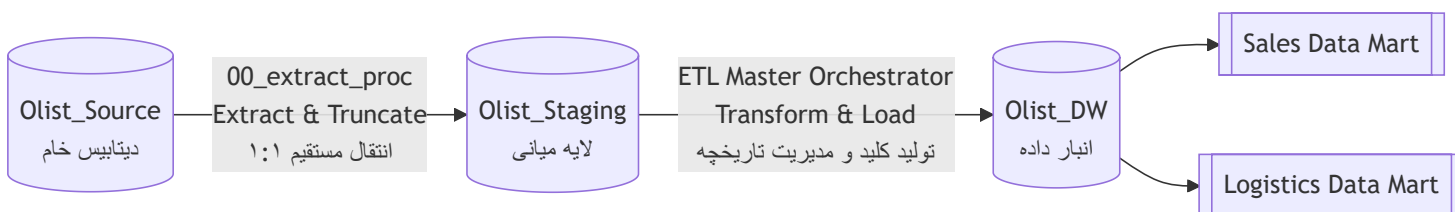
فهرست مطالب

- ۰. مقدمه و دامنه سند
- فلوچارت کلی پایپ‌لاین ETL
- ۱. نگاشت از لایه منبع به استیجینگ
- ۲. نگاشت ابعاد انبار داده
- ۳. نگاشت فکت‌ها و دیتا مارت‌ها
- ۴. کنترل کیفیت داده و مدیریت خطا

۰. مقدمه و دامنه سند

این مستند قوانین استخراج، تبدیل و بارگذاری (ETL) را که برای انتقال داده‌ها از سیستم عملیاتی خام (Source) به لایه میانی (Staging) و در نهایت به انبار داده (Data Warehouse) اعمال شده‌اند، تشریح می‌کند. این سند مکمل `Data_Warehouse_Design.md` است و باید هم‌زمان با آن مطالعه شود، چون طراحی ابعاد (Dimension) و فکت‌ها در آن سند تعریف شده و منطق بارگذاری‌شان اینجا آمده است.

فلوچارت کلی پایپ‌لاین ETL



۱. نگاشت از لایه منبع به استیجینگ (Source to Staging)

- استراتژی: استخراج کامل (Full Extract) و Truncate-Load
- هدف: استخراج داده‌ها از سیستم عملیاتی و انتقال به محیط Staging بدون اعمال تغییرات سنگین، جهت به حداقل رساندن بار پردازشی (Overhead) روی سیستم مبدأ.
- رویه اجرا: `extract_proc_00` (اجرای دستی/زمان‌بندی‌شده پیش از هر بار اجرای Orchestrator اصلی)

جدول هدف در لایه	جدول مبدأ در	عملیات / منطق انتقال (Transformation)
<code>Olist_Staging</code>	<code>Olist_Source</code>	
<code>stg_customers</code>	<code>customers</code>	درج مستقیم داده‌ها (Direct Insert 1:1)
<code>stg_orders</code>	<code>orders</code>	درج مستقیم داده‌ها (Direct Insert 1:1)
<code>stg_order_items</code>	<code>order_items</code>	درج مستقیم داده‌ها (Direct Insert 1:1)
<code>stg_sellers</code>	<code>sellers</code>	درج مستقیم داده‌ها (Direct Insert 1:1)
<code>stg_products</code>	<code>products</code>	درج مستقیم. هندل کردن خطای تایپی <code>length</code> در دیتاست اصلی Kaggle (تغییر نام ستون به <code>product_length_cm</code> هنگام درج).
<code>stg_reviews</code>	<code>reviews</code>	درج مستقیم داده‌ها (Direct Insert 1:1)

نکات پیاده‌سازی:

- هیچ فیلتر یا Deduplication در این مرحله اعمال نمی‌شود؛ پاکسازی و رفع تکرار به مرحله‌ی بارگذاری ابعاد/فکت‌ها موکول شده است.
- ستون‌های Staging از نوع داده‌ی متن (`nvarchar` / `varchar`) نگهداری می‌شوند تا خطای Type Conversion در حین Extract رخ ندهد؛ Cast نهایی در لایه DW انجام می‌گیرد.

۲. نگاشت ابعاد انبار داده (Staging to Dimensions)

- استراتژی: تخصیص کلید جایگزین (Surrogate Key) و مدیریت ابعاد به کندی متغیر (SCD).

منطق نگاشت و تبدیل	نوع SCD	جدول لایه استیجینگ	جدول بُعد (Dimension)
نگاشت داده‌های جغرافیایی مشتری (, customer_city customer_state). تغییر در شهر یا استان باعث بسته‌شدن سطر جاری (, ValidTo = GETDATE()) (IsCurrent = 0) و درج یک سطر جدید (IsCurrent = 1 , ValidTo = NULL) می‌شود.	Type 2	stg_customers	dim_customer
نگاشت ویژگی‌های محصول. در صورت تغییر دسته‌بندی، سطر جدید ایجاد نمی‌شود؛ مقدار قبلی در ستون product_category_name_previous نگهداری و مقدار جدید در product_category_name جایگزین می‌شود (فقط یک نسل تاریخچه قابل بازیابی است).	Type 3	stg_products	dim_product
به‌روزرسانی اطلاعات فروشنده با استفاده از دستور MERGE . داده‌های جدید جایگزین قبلی می‌شوند (بدون حفظ تاریخچه).	Type 1	stg_sellers	dim_seller
نگاشت مستقیم بدون تغییر. به‌منظور حفظ یکپارچگی ارجاعی در fact_logistics ، یک رکورد پیش‌فرض با شناسه‌ی review_sk = -1 برای سفارش‌های فاقد نظر ثبت‌شده توزیع می‌شود.	Type 0 (Fixed)	stg_reviews	dim_review
تولید خودکار توسط حلقه‌ی WHILE در T-SQL برای بازه‌ی ۲۰۱۶ تا ۲۰۱۹. محاسبه‌ی is_weekend با تابع . DATEPART(WEEKDAY, ...)	N/A	تولید رویه‌ی	dim_date

محدودیت شناخته‌شده: dim_date تعطیلات رسمی برزیل را محاسبه نمی‌کند؛ فقط آخر هفته (is_weekend) از طریق DATEPART استخراج می‌شود. اگر تحلیل تاخیر تحویل نیاز به لحاظ‌کردن تعطیلات رسمی داشته باشد، باید یک جدول مرجع تعطیلات به‌صورت جداگانه بارگذاری و به dim_date متصل (Join) شود.

۳. نگاشت فکت‌ها و دیتا مارت‌ها (Staging to Facts)

- استراتژی: بارگذاری افزایشی تکرارپذیر (Idempotent Incremental Load) و اعمال کلیدهای خارجی (Foreign Keys).

منطق نگا	گرین (Grain)	جداول لایه استیجینگ	جدول فکت
استخراج SK از <code>ate_sk</code> به <code>order_purchase_timestamp</code> استفاده <code>NOT EXISTS (order_id, order_item_id)</code> برای جلوگیری از درج رکوردهای تکراری	یک سطر به ازای هر <code>order_id + order_item_id</code>	<code>stg_orders</code> , <code>stg_order_items</code>	<code>fact_sales</code>
محاسبه‌ی تاخیر تحویل <code>DATEDIFF(day, estimated, actual)</code> مقادیر NULL در <code>seller_sk</code> یا <code>review_sk</code> <code>NOT EXISTS (order_id)</code> استفاده از -1 برای تضمین <code>accuracy</code>	یک سطر به ازای هر <code>order_id</code> (تجمیع‌شده در سطح سفارش)	<code>stg_orders</code> , <code>stg_order_items</code>	<code>fact_logistics</code>

توجه به تفاوت گرین: `fact_sales` در سطح کالای سفارش (order item) است ولی `fact_logistics` در سطح سفارش (order) است — چون تاخیر تحویل و هزینه‌ی حمل معمولاً به کل سفارش مربوط می‌شود، نه به هر قلم کالا. هنگام نوشتن کوئری‌های مقایسه‌ای بین دو مارت، این تفاوت گرین باید مدنظر باشد تا از Double Counting جلوگیری شود.

۴. کنترل کیفیت داده و مدیریت خطا

نحوه‌ی مدیریت	بررسی
جایگزینی با رکورد -1 («نامشخص») در بُعد مربوطه	کلید خارجی گم‌شده (مثلاً <code>seller_id</code>) بدون رکورد متناظر)
Rename در حین Extract به <code>product_length_cm</code>	خطای نام ستون در منبع خام (<code>length</code>)
<code>delivery_delay_days</code> به‌صورت NULL باقی می‌ماند (نه صفر) تا با میانگین‌گیری اشتباه ترکیب نشود	تاریخ تحویل خالی (سفارش لغوشده/ در حال ارسال)
استفاده از بند <code>WHERE NOT EXISTS</code> در تمام <code>Insert</code> های فکت، برای تضمین Idempotency	اجرای مجدد پایپ‌لاین (Re-run)